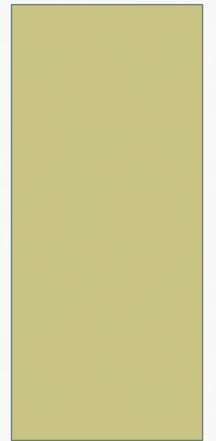


FORTRAN 90

CONCEPTOS BÁSICOS



INTRODUCCIÓN

- **FORTRAN** es un lenguaje de programación de **alto-nivel**, es decir, necesita de un **compilador** para traducir las operaciones realizadas, éstas regularmente escritas en un lenguaje que el usuario puede utilizar más fácilmente; el **compilador** traduce a **lenguaje-máquina** para que la computadora pueda entender qué esperamos que realice.
- **FORTRAN** ha estado en la industria desde hace más de 50 años y es especialmente útil en el Análisis numérico y en los cálculos técnicos.
- El nombre **FORTRAN** deriva de **FOR**mula **TRAN**slation, entendiéndose que el desarrollo del lenguaje tenía la intención desde un principio de traducir ecuaciones científicas a un código computacional.
- Las versiones subsiguientes de **FORTRAN** fueron:
 - FORTRAN IV, FORTRAN 66, FORTRAN 77, **Fortran 90**, Fortran 95, Fortran 2003, Fortran 2008.

COMPILADORES FORTRAN

- Los principales compiladores de **FORTRAN** son:

- F2c
- G95
- **Gfortran**
- GNU Compiler Collection
- Intel Fortran Compiler
- Numerical Algorithms Group
- Open64
- Oracle Solaris Studio
- PathScale
- The Portland Group
- Silverfrost FTN95
- IBM VisualAge

La mayoría desarrollados por compañías que poseen los derechos de patente de su compilador respectivo, y algunos de uso libre como es el compilador **Gfortran**, del cual utilizaremos la versión FORTRAN 90 por su característica de portabilidad a las demás versiones anteriores así como posteriores.

ESTRUCTURA DEL PROGRAMA

- Un programa de Fortran 90 tiene la forma siguiente:

```
PROGRAM nombre-programa
    IMPLICIT NONE
    [Seccion de especificaciones]
    [Seccion de ejecucion]
    [Seccion de subprogramas]
END OF PROGRAM nombre-programa
```

- **nombre-programa** es el nombre del programa.
- **Sección de especificaciones**, **Sección de ejecución**, y la **Sección de subprogramas** son opcionales.
- Aunque **IMPLICIT NONE** es también opcional, es requerido para escribir programas seguros.

COMPILANDO, ENLAZANDO Y EJECUTANDO UN PROGRAMA

- Antes de que un programa de Fortran 90 pueda ser ejecutado, éste debe ser compilado, y luego enlazado a las librerías de la computadora para producir un programa ejecutable.



```
Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Reservados todos los derechos.

C:\>gfortran nombre-programa.f90 -o nombre-programa_
```

The screenshot shows a Windows command prompt window titled 'Administrator: C:\Windows\system32\cmd.exe'. The window displays the Microsoft Windows version (6.1.7601) and copyright information (© 2009 Microsoft Corporation). The command prompt shows the command `C:\>gfortran nombre-programa.f90 -o nombre-programa_` being entered.

COMENTARIOS EN FORTRAN

- Los Comentarios comienzan con el símbolo de exclamación !
- Todo lo escrito después de ese símbolo será ignorado por el compilador.
- Es similar a // en C/C++

```
PROGRAM          ComentariosPrueba1
IMPLICIT NONE
REAL :: Year ! Define como real el valor de Año
!.....
PRINT *, "Introduzca el valor del año:"
READ *, Year  ! lee el valor de Año
!.....
Year = Year + 1  ! incrementa en 1 a Año
!.....
PRINT *, Year  ! imprime el nuevo valor de Año
END PROGRAM    ComentariosPrueba1
```

LÍNEAS DE CONTINUACIÓN/ALFABETOS

- Si la declaración es demasiado larga para ajustarse a una sola línea, ésta debe **continuar** con el caracter de continuación **&**, el cual no es parte de la declaración.

```
Total = Total + &  
                Monto * Pagos  
! Total = Total + Monto * Pagos  
PROGRAM &  
                linea-continuacion  
END OF PROGRAM linea-continuacion
```

- Los caracteres en Fortran 90 son los siguientes:

- Letras mayúsculas y minúsculas.
- Dígitos.
- Caracteres especiales.

espacio ' " () * + - / : = _ !
& \$; < > % ? , .

CONSTANTES 1/2

- Una constante en Fortran 90 puede ser: **INTEGER**, **REAL**, **LOGICAL**, **COMPLEX**, y **CHARACTER STRING**.
- Una constante **INTEGER** es una cadena de dígitos con un signo opcional: **12345**, **-345**, **+789**, **+9**.
- Una constante **REAL** tiene dos formas **decimal** y **exponencial**:
 - En la forma **decimal**, una constante real es una cadena de dígitos con un punto decimal. Una constante real también puede incluir un signo opcional.

Por ejemplo: **2.45**, **.13**, **13.**, **-0.12**, **- .15**.

- En la forma **exponencial**, una constante real comienza con un integer/real, seguido por una E/e, seguido de un entero, esto es el exponente.

Por ejemplo:

12E3 = 12×10^3 , **3.45E-8** = 3.45×10^{-8}
-12E3 = -12×10^3 , **-3.45E-8** = -3.45×10^{-8}

CONSTANTES 2/2

- Una constante **LOGICAL** es ya sea **.TRUE.** o **.FALSE.**
- Nótese que los periodos alrededor de **TRUE** y **FALSE** son requeridos
- Una cadena de caracteres o **CHARACTER STRING** se encuentra siempre dentro de dos comillas o comillas simples. Por ejemplo: **"abc"**, **'Juan Perez'**, **"#\$%&"**.
- El contenido de una cadena de caracteres consta de todos los caracteres entre las comillas. Por ejemplo: **'Juan Perez'** es **Juan Perez**.
- La longitud de una cadena es el número de caracteres entre las comillas. Por ejemplo: la longitud de **'Juan Perez'** es 10, contando el espacio en blanco.

DECLARACIONES 1/2

- Fortran 90 utiliza la siguiente forma para declarar variables,
Especificador-tipo :: lista
- Donde **Especificador-tipo** es alguna de las siguientes variables: **INTEGER**, **REAL**, **LOGICAL**, **COMPLEX** y **CHARACTER**; y **lista** es una secuencia de identificadores o nombres de variables utilizadas en el programa, separados por comas:
- Por ejemplo:

```
INTEGER :: postal, Total, contador  
REAL    :: PROMEDIO, x, Diferencia  
LOGICAL :: Condicion, OK  
COMPLEX :: Conjugado
```

DECLARACIONES 2/2

- Las variables tipo **CHARACTER** requieren de información adicional, la longitud de la cadena
 - A **CHARACTER** debe de seguir el atributo longitud (**LEN = 1**), donde **1** es la longitud de la cadena.
 - Por ejemplo:

```
CHARACTER(LEN=20)  :: Respuesta, Pregunta
!Las variables Respuesta y pregunta pueden
!contener hasta 20 caracteres
CHARACTER(20)     :: Respuesta, Pregunta
!Cumple con las mismas propiedades que la
!anterior declaración.
CHARACTER :: Tecla
!Significa que la variable Tecla puede
!contener solo un caracter.
```

OPERADOR CHARACTER

- Fortran utiliza `//` para concatenar dos cadenas.
- Si la cadena **A** y la cadena **B** tienen una longitud m y n , la concatenación **A** `//` **B** es una cadena de longitud $m + n$

```
CHARACTER(LEN=4) :: Juan = " Juan", Sol = "Sol"  
CHARACTER(LEN=6) :: Laura = "Laura", Renata = "Renata"  
CHARACTER(LEN=10) :: Ans1, Ans2, Ans3, Ans4
```

Ans1 = Juan // Laura	! Ans1 = "JuanLaura-"
Ans2 = Sol // Renata	! Ans2 = "Sol-Renata"
Ans3 = Renata // Sol	! Ans3 = "RenataSol-"
Ans4 = Laura // Sol	! Ans4 = "Laura-Sol-"

EL ATRIBUTO PARÁMETRO

- Un identificador tipo **PARAMETER** es un nombre cuyo valor no puede ser modificado. En otras palabras es una constante.
- El atributo **PARAMETER** se usa después del tipo de variable.
- Cada identificador es seguido por un **=** y el valor del identificador. Por ejemplo:

```
INTEGER, PARAMETER :: MAXIMO = 10  
REAL, PARAMETER :: PI = 3.1415926, E = 2.17828  
LOGICAL, PARAMETER :: TRUE = .true., FALSE = .false.
```

- Se puede inicializar variables de tres formas:
 - Inicialización: Se realiza una vez que el programa se ejecuta.
 - Asignación: Se realiza cuando el programa ejecuta una declaración asignada.
 - Entrada: Es cuando se lee la variable con la función **READ**.

OPERADORES ARITMÉTICOS

- Hay cuatro tipos de operadores en Fortran 90: aritméticos, relacionales, lógicos y de caracteres.

Tipo	Operador		Asociatividad
Aritméticos	**		Derecha a izquierda
	*	/	Izquierda a derecha
	+	-	Izquierda a derecha
Relacionales	< .LT. <= .LE. > .GT.	>= .GE. == .EQ. /= .NE.	Ninguna
Lógicos	.NOT.		Derecha a izquierda
	.AND.		Izquierda a derecha
	.OR.		Izquierda a derecha
	.EQV.	.NEQV.	Izquierda a derecha

FUNCIONES INTRÍNSECAS DE FORTRAN

- Para utilizar una función intrínseca en fortran necesitamos saber:
 - Nombre y significado de la función.
 - Número de argumentos.
 - El tipo y el rango de cada argumento.
 - El tipo de variable que devuelve la función.

Función	Significado	Entrada	Devuelve
ABS(X)	Valor absoluto de x	INTEGER	INTEGER
SQRT(X)	Raíz cuadrada de x	REAL	REAL
SIN(X)	Seno de x en radianes	REAL	REAL
COS(X)	Coseno de x en radianes	REAL	REAL
TAN(X)	Tangente de x en radianes	REAL	REAL
ASIN(X)	Arco seno de x	REAL	REAL
ACOS(X)	Arco coseno de x	REAL	REAL
ATAN(X)	Arco tangente de x	REAL	REAL
EXP(X)	Exponencial de x e^x	REAL	REAL
LOG(X)	Logaritmo natural de x	REAL	REAL
LOG10(X)	Logaritmo común de x	REAL	REAL

FUNCIONES INTRÍNSECAS DE FORTRAN

- Algunas funciones de conversión:

Función	Significado	Entrada	Devuelve
INT(X)	Trunca a la parte entera de x	INTEGER	INTEGER
NINT(X)	Redondea al entero más cercano a x	REAL	REAL
FLOOR(X)	Redondea al mayor entero menor que o igual al valor de x	REAL	REAL
FRACTION(X)	Trunca a la parte fraccional de x	REAL	REAL
REAL(X)	Convierte a x a REAL	REAL	REAL
MAX(x1,x2,...,xn)	Máximo de x1,x2,...,xn	INTEGER/REAL	INTEGER/REAL
MIN(x1,x2,...,xn)	Mínimo de x1,x2,...,xn	INTEGER/REAL	INTEGER/REAL
MOD(x,y)	Residuo de x- INT(x/y)*y	INTEGER/REAL	INTEGER/REAL

- Algunos ejemplos:

INT(-3.5) devuelve -3
NINT(3.5) devuelve 4
NINT(-3.4) devuelve -3
FLOOR(3.6) devuelve 3

FLOOR(-3.5) devuelve -4
FRACTION(12.3) dev. 0.3
REAL(-10) dev. -10.0

¿QUE ES **IMPLICIT NONE**?

- Fortran tiene una interesante tradición: todas las variables que comienzan con **i, j, k, l, m** y **n**, si no son declaradas, son del tipo **INTEGER** por defecto. Esta característica puede resultar en problemas de definición si no se maneja con cuidado.
- **IMPLICIT NONE** significa que todos los nombres deben ser declarados y que no hay ningún **INTEGER** asumido implícitamente.
- Todos nuestros programas por seguridad deben de contener ésta declaración.

DECLARACIÓN READ

- Fortran 90 usa la declaración `READ(*,*)` para leer datos que asignará a variables desde el teclado, por ejemplo:

```
INTEGER :: Edad !Se inicializan las variables  
REAL :: Cantidad, Promedio  
CHARACTER(LEN=10) :: Nombre
```

```
READ(*,*) Nombre, Edad, Promedio, Cantidad
```

- `READ(*,*)` lee datos desde el teclado por defecto.
- Si `READ(*,*)` tiene n variables, debe de haber n constantes declaradas.
- Cada variable debe de ser declarada de acuerdo a su tipo correspondiente. Los enteros pueden ser leídos como reales pero no vice versa.
- Los elementos leídos se separan mediante comas y espacios y pueden esparcirse por varias líneas.

DECLARACIÓN WRITE

- Fortran 90 utiliza la declaración `WRITE(*,*)` para escribir información en la pantalla.
- `WRITE(*,*)` tiene dos formas, donde `exp1` `exp2` y `exp3` son expresiones anteriormente declaradas.

```
WRITE(*,*) exp1, exp2, exp3 !Imprime en pantalla los valores
WRITE(*,*)                  !Imprime una línea en blanco
```

- `WRITE(*,*)` evalúa el resultado de cada expresión y lo imprime en la pantalla.
- `WRITE(*,*)` puede escribir texto predeterminado en la pantalla.

```
WRITE(*,*) 'El Valor de exp1 es =', exp1
```

EJEMPLO COMPLETO

- Este programa debe de calcular la posición (coordenadas x y y) y la velocidad (magnitud y dirección) de un proyectil, dados t , el tiempo desde el lanzamiento, u , la velocidad del lanzamiento, a , el ángulo inicial de lanzamiento en grados, y $g=9.8$, debido a la aceleración de la gravedad.

- Los desplazamientos horizontales y verticales x y y se obtienen de la forma siguiente:

$$x = u * \cos(a) * t$$
$$y = u * \sin(a) * t - \frac{g * t^2}{2}$$

- Las componentes vertical y horizontal del vector velocidad se obtienen de la forma siguiente:

$$V_x = u * \cos(a)$$
$$V_y = u * \sin(a) - g * t$$

EJEMPLO COMPLETO

- La magnitud del vector velocidad se obtiene de la forma siguiente:

$$V = \sqrt{V_x^2 + V_y^2}$$

- El ángulo entre el suelo y el vector velocidad se obtiene de la forma siguiente:

$$\tan(\theta) = \frac{V_x}{V_y}$$

EJEMPLO COMPLETO

```
PROGRAM Proyectil
  IMPLICIT NONE
  REAL, PARAMETER :: g = 9.8          ! Aceleración debido a la gravedad
  REAL, PARAMETER :: PI = 3.1415926  ! Declaración del valor de pi
  REAL :: Angulo                      ! Angulo de lanzamiento en grados
  REAL :: Tiempo                      ! Tiempo desde el lanzamiento
  REAL :: Theta                       ! Angulo entre el vector velocidad y el suelo en grados
  REAL :: U                           ! Velocidad de lanzamiento
  REAL :: V                           ! Velocidad resultante
  REAL :: Vx                          ! Velocidad horizontal
  REAL :: Vy                          ! Velocidad vertical
  REAL :: X                           ! Desplazamiento horizontal
  REAL :: Y                           ! Desplazamiento vertical

  WRITE(*,*) 'Introduzca los valores de: angulo de lanzamiento en grados, &
             tiempo desde el lanzamiento y la velocidad de lanzamiento:'
  READ (*,*) Angulo, Tiempo, U
```

EJEMPLO COMPLETO

```
Angulo = Angulo * PI / 180.0          ! Convirtiendo a radianes
X = U * COS(Angulo) * Tiempo
Y = U * SIN(Angulo) * Tiempo - g*Tiempo**2 / 2.0
Vx = U * COS(Angulo)
Vy = U * SIN(Angulo) - g * Tiempo
V = SQRT(Vx**2 + Vy**2)
Theta = ATAN(Vy/Vx) * 180.0 / PI      ! Convirtiendo a grados

WRITE(*,*) , Angulo
WRITE(*,*) 'Desplazamiento Horizontal : ', X
WRITE(*,*) 'Desplazamiento Vertical : ', Y
WRITE(*,*) 'Velocidad Resultante : ', V
WRITE(*,*) 'Direccion (en grados) : ', Theta

END PROGRAM Proyectil
```